

Assignment-17 (Lab-17.2)

Course : AI Assisted Coding

Topic: AI for Data Processing

Student Details

Name :Batti Chandan Singh

Hall Ticket :2303A51515

Batch :22

Task-01

Final Optimal Prompt

Generate a Python script using Pandas to clean student academic data.

Handle missing values in marks, attendance, and department.

Remove duplicate records based on student_id.

Standardize department names, encode categorical variables like gender and department, and handle invalid inputs safely.

Code Screenshot

```
import pandas as pd

data = pd.read_csv("D:\\6th Sem 2025-26\\AI-Assisted Coding\\Assignments python file\\Assignment-17(Lab-17.2)\\students.csv")

# Handle missing values
data['marks'].fillna(data['marks'].mean(), inplace=True)
data['attendance'].fillna(data['attendance'].mean(), inplace=True)
data['department'].fillna("Unknown", inplace=True)

# Remove duplicates
data.drop_duplicates(subset='student_id', inplace=True)

# Standardize text
data['department'] = data['department'].str.lower().str.strip()

# Encode categorical data
data['gender'] = data['gender'].astype('category').cat.codes
data['department'] = data['department'].astype('category').cat.codes

print(data.head())
```

Output Screenshot

	student_id	name	marks	attendance	department	gender
0	1	Arjun	85.0	90.0	0	1
1	2	Meera	NaN	88.0	2	0
2	3	Rahul	72.0	NaN	0	1
3	4	Anita	95.0	92.0	1	0

PS D:\6th Sem 2025-26\AI-Assisted Coding\Assignments python file> | In 10.4

Explanation/Justification/Observation (100 words / 5 – 6 sentence)

This task focuses on cleaning student academic data using Pandas. Missing values in marks and attendance are replaced using mean values, while missing department values are filled with "Unknown". Duplicate records are removed using student_id to ensure data uniqueness. Text fields such as department names are standardized by converting them to lowercase and removing extra spaces. Categorical variables like gender and department are encoded into numerical values for analysis. This preprocessing step ensures the dataset is clean, consistent, and ready for machine learning or analysis. It improves data quality and prevents errors during further processing or model training.

Task-02

Final Optimal Prompt

Generate a Python script to preprocess financial transaction data.

Convert date columns to datetime, extract month and year, normalize transaction amount, and handle extreme values using appropriate methods.

Code Screenshot

```
32 #---Lab-17.2-Task-02/05---
33 '''Generate a Python script to preprocess financial transaction data.
34 Convert date columns to datetime, extract month and year, normalize transaction amount,
35 and handle extreme values using appropriate methods.'''
36 import pandas as pd
37 from sklearn.preprocessing import MinMaxScaler
38
39 transactions = pd.read_csv("D:\\6th Sem 2025-26\\AI-Assisted Coding\\Assignments python file\\Assignment-17
40
41 # Convert to datetime
42 transactions['date'] = pd.to_datetime(transactions['date'])
43
44 # Extract features
45 transactions['month'] = transactions['date'].dt.month
46 transactions['year'] = transactions['date'].dt.year
47
48 # Normalize amount
49 scaler = MinMaxScaler()
50 transactions['amount'] = scaler.fit_transform(transactions[['amount']])
51
52 # Handle outliers
53 transactions = transactions[transactions['amount'] < 0.95]
54
55 print(transactions.head())
56
```

Output Screenshot

```
transaction_id  amount      date  month  year
0              1  0.072165 2024-01-10      1  2024
1              2  0.226804 2024-02-15      2  2024
2              3  0.000000 2024-03-01      3  2024
3              4  0.432990 2024-03-20      3  2024
PS D:\6th Sem 2025-26\AI-Assisted Coding\Assignments python file>
```

Explanation/Justification/Observation (100 words / 5 – 6 sentence)

This task preprocesses financial transaction data using Python and Pandas. The date column is converted into datetime format to enable time-based operations. New features such as transaction month and year are extracted for better analysis. The transaction amount is normalized using MinMaxScaler to bring values within a consistent range. Extreme values or outliers are filtered to improve data quality and avoid skewed results. This preprocessing step helps in preparing data for machine learning models or financial analysis. It ensures

consistency, reduces noise, and enhances the ability to detect patterns and trends in transaction data effectively.

Task-03

Final Optimal Prompt

Generate a Python script to clean environmental sensor data.

Handle missing values, scale numerical features, encode categorical fields, and split dataset into training and testing sets.

Code Screenshot

```
65 import pandas as pd
66 from sklearn.preprocessing import StandardScaler
67 from sklearn.model_selection import train_test_split
68
69 data = pd.read_csv("D:\\6th Sem 2025-26\\AI-Assisted Coding\\Assignments python file\\Assignment
70
71 # Handle missing values
72 data.fillna(data.mean(numeric_only=True), inplace=True)
73
74 # Encode categorical
75 data['sensor_status'] = data['sensor_status'].astype('category').cat.codes
76
77 # Scale features
78 scaler = StandardScaler()
79 data[['temperature', 'humidity', 'air_quality_index']] = scaler.fit_transform(
80 | data[['temperature', 'humidity', 'air_quality_index']]
81 )
82
83 # Split data
84 train, test = train_test_split(data, test_size=0.2, random_state=42)
85
86 print(train.head())
87
```

Output Screenshot

```
temperature  humidity  air_quality_index  sensor_status
4      0.622841    0.667859           1.700840             1
2     -1.453296    0.000000          -0.566947             0
0     -0.622841   -1.706750           0.188982             0
3      1.453296    1.261511           0.000000             0
PS D:\6th Sem 2025-26\AI-Assisted Coding\Assignments python file>
```

Explanation/Justification/Observation (100 words / 5 – 6 sentence)

This task prepares environmental sensor data for predictive modeling. Missing values are handled using mean imputation to maintain dataset consistency. Categorical fields like sensor status are encoded into numeric values for model compatibility. Numerical features such as temperature, humidity, and air quality index are scaled using StandardScaler to

standardize their ranges. The dataset is then split into training and testing subsets to support machine learning workflows. This preprocessing ensures that the data is clean, normalized, and suitable for model training. It improves accuracy and prevents bias caused by inconsistent or missing data values.

Task-04

Final Optimal Prompt

Generate a Python script to clean smart city traffic data.

Standardize text fields, handle missing values, remove duplicates,

normalize numeric columns, and generate a summary of data quality improvement.

Code Screenshot

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

traffic = pd.read_csv("D:\\6th Sem 2025-26\\AI-Assisted Coding\\Assignments python file\\Assignn

before = traffic.describe()

# Clean data
traffic['road'] = traffic['road'].str.lower().str.strip()
traffic['location'] = traffic['location'].str.lower().str.strip()

traffic.fillna(traffic.mean(numeric_only=True), inplace=True)
traffic.drop_duplicates(inplace=True)

scaler = MinMaxScaler()
traffic[['speed', 'vehicle_count']] = scaler.fit_transform(
    traffic[['speed', 'vehicle_count']]
)

after = traffic.describe()

print("Before Cleaning:\n", before)
print("After Cleaning:\n", after)
```

Output Screenshot

```
Before Cleaning:
      traffic_density      speed  vehicle_count
count      4.000000      5.000000      5.000000
mean      65.000000     48.400000     282.000000
std      17.320508     10.620734     107.796104
min      50.000000     40.000000     200.000000
25%      50.000000     40.000000     200.000000
50%      65.000000     42.000000     210.000000
75%      80.000000     60.000000     400.000000
max      80.000000     60.000000     400.000000
```

```
After Cleaning:
      traffic_density      speed  vehicle_count
count      3.0      3.000000      3.000000
mean      65.0      0.366667      0.350000
std      15.0      0.550757      0.563471
min      50.0      0.000000      0.000000
25%      57.5      0.050000      0.025000
50%      65.0      0.100000      0.050000
75%      72.5      0.550000      0.525000
max      80.0      1.000000      1.000000
```

```
○ PS D:\6th Sem 2025-26\AI-Assisted Coding\Assignments python file> █
```

Explanation/Justification/Observation (100 words / 5 – 6 sentence)

This task focuses on cleaning smart city traffic data. Text fields like road and location names are standardized to ensure consistency. Missing numerical values are filled using mean values to maintain dataset completeness. Duplicate records are removed to avoid redundancy. Numerical features such as speed and vehicle count are normalized using MinMaxScaler for better analysis. A summary is generated before and after cleaning to compare improvements in data quality. This process ensures the dataset is accurate and reliable. Clean traffic data is essential for smart city applications like traffic prediction, congestion analysis, and urban planning systems.

Task-05

Final Optimal Prompt

Generate a Python script to clean social media text data.

Remove duplicates, clean text (URLs, symbols), convert to lowercase,

remove stopwords, perform tokenization, and extract features like word count.

Code Screenshot

```
17 #---Lab-17.2-Task-05/05---
18 '''Generate a Python script to clean social media text data.
19 Remove duplicates, clean text (URLs, symbols), convert to lowercase,
20 remove stopwords, perform tokenization, and extract features like word count.'''
21 import pandas as pd
22 import re
23
24 posts = pd.read_csv("D:\\6th Sem 2025-26\\AI-Assisted Coding\\Assignments python file\\Assignment-17(Lab-17.2).py")
25
26 # Remove duplicates & nulls
27 posts.drop_duplicates(inplace=True)
28 posts.dropna(subset=['text'], inplace=True)
29
30 def clean_text(text):
31     text = re.sub(r"http\S+", "", text)
32     text = re.sub(r"[^a-zA-Z ]", "", text)
33     return text.lower().strip()
34
35 posts['clean_text'] = posts['text'].apply(clean_text)
36
37 # Feature extraction
38 posts['word_count'] = posts['clean_text'].apply(lambda x: len(x.split()))
39
40 print(posts.head())
41
```

Output Screenshot

```
● PS D:\6th Sem 2025-26\AI-Assisted Coding\Assignments python file> & "D:\python 3.13.6\python.exe" "d:/6th Sem 2025-26/AI-Assisted Coding/Assignments python file/Assignment-17(Lab-17.2).py"
  post_id      text      clean_text  word_count
0        1  check this out http://example.com  check this out         3
1        2    Amazing product!!!    amazing product         2
3        4    Worst experience ever!!!  worst experience ever         3
○ PS D:\6th Sem 2025-26\AI-Assisted Coding\Assignments python file> █
```

Explanation/Justification/Observation (100 words / 5 – 6 sentence)

This task cleans and preprocesses social media text data for sentiment analysis. Duplicate and null entries are removed to ensure data quality. Text is cleaned by removing URLs, special characters, and extra spaces. All text is converted to lowercase for uniformity. Basic feature extraction is performed by calculating word count. These preprocessing steps help transform raw text into structured data suitable for analysis. Cleaned text improves the performance of natural language processing models. This task demonstrates how AI-assisted scripting can automate repetitive text cleaning tasks efficiently and prepare data for machine learning applications.